

Trabajo Práctico Final

Estrategia de Testing

Aplicación: Clínica

1. Presentación y Objetivos

El presente trabajo práctico tiene como propósito demostrar que el estudiante a adquirido la capacidades básicas para el testing de un producto de Software. El desarrollo exige estructurar las validaciones diferenciando estrictamente dos niveles arquitectónicos: el **testing atómico (unitario)** sobre componentes independientes y el **testing de integración** sobre componentes acoplados con recursos externos (bases de datos).

2. Consignas

1) Testing de la clase AsociadoDTO

- ▶ Desarrollar una suite de pruebas unitarias que compruebe de forma estrictamente aislada la correcta asignación, recuperación y las validaciones internas de integridad implementadas en los *setters* del objeto de transferencia de datos.
- ▶ **Validación:** Ejecutar la suite de pruebas y demostrar que es capaz de detectar los fallos o inconsistencias lógicas que pudieran existir en el código provisto, sin necesidad de levantar servicios externos.
- ▶ Incluir las conclusiones en el informe final y lo resultados en el apéndice.

2) Testing de Integración

Corregir si es necesario el código del item 1 y realizar la prueba de los módulos: **AsociadoDTO, DataAccessObject y BaseDeDatos.**

- ▶ Desarrollar una suite de pruebas de integración orientada a verificar la comunicación real con el motor de base de datos **MySQL**, validando la correcta ejecución del patrón Singleton en la conexión, la inicialización automática de tablas y los datos semilla, así como las operaciones **CRUD** a través de consultas con parametros.
- ▶ **Validación:** Ejecutar las pruebas de integración contra la base de datos y demostrar cómo la suite atrapa las excepciones de SQL, los errores de mapeo o las inconsistencias que pudieran existir en el código provisto."
- ▶ Incluir las conclusiones en el informe final y lo resultados en el apéndice.

3) Testing de integración con Mocks.

Para este test se utilizarán la clase **Ambulancia**, la interfaz **State** y la rutina concurrente **RetornoAutomarico**.

- ▶ Testing con Mocks: Aprovechar el diseño desacoplado basado en la interfaz **State** para construir pruebas atómicas utilizando dobles de prueba (mocks), simulando comportamientos de estados ante la ausencia temporal de las implementaciones concretas.
- ▶ Testing de Integración y Concurrencia: Evaluar el flujo completo de la máquina de estados acoplada a la clase **Ambulancia**, validando la correcta gestión de recursos compartidos, bloques sincronizados y la interacción con hilos independientes de ejecución.
- ▶ **Validación:** Demostrar mediante las pruebas de integración y el uso de mocks que la suite descubre los problemas de sincronización o fallos de estado introducidos en el sistema.

3. Criterios de Entrega

- Código fuente completo de las suites de pruebas implementadas.
- Capturas de ejecución mostrando las pruebas en verde (funcionamiento correcto) y en rojo (detección de los errores).
- Informe documentando el diseño formal de escenarios, casos de prueba (con su respectiva aplicación de clases de equivalencia y valores límite), las conclusiones y los resultados obtenidos